

Apply filters to SQL queries

Project description

My organization is working to make their system more secure. It is my job to ensure the system is safe, investigate all potential security issues, and update employee computers as needed. The following steps provide examples of how I used SQL with filters to perform security-related tasks.

Retrieve after hours failed login attempts

There was a security incident that occurred after business hours (after 18:00). We will investigate all after hour logins.

The following screenshot shows the code i used to demonstrates the SQL query filters i used to filter for the logins that were made after business hours.

```
MariaDB [organization]> SELECT *  
-> FROM log_in_attempts  
-> WHERE login_time > '18:00' AND success = FALSE;
```

event_id	username	login_date	login_time	country	ip_address	success
2	apatel	2022-05-10	20:27:27	CAN	192.168.205.12	0
18	pwashing	2022-05-11	19:28:50	US	192.168.66.142	0
20	tshah	2022-05-12	18:56:36	MEXICO	192.168.109.50	0

The first part of the screenshot is the query I used, and the second part is the output. This query filters for login attempts made after 18:00. First I used the `SELECT` and `FROM` clause to select all data in `log_in_attempts`. Then, i used the `WHERE` clause with an AND operator to filter my results to only show attempts that happened after 18:00 and were not successful. The conditional expression `login_time > '18:00'` is what filters for the login attempts that occurred after 18:00. The condition is `success = FALSE`, which filters for the failed login attempts.

Retrieve login attempts on specific dates

A suspicious event occurred on 2022-05-09. I need to retrieve all log in attempts that occurred on 2022-05-09 and the day before which is 2022-05-08.

The following code demonstrates how I created a SQL query to filter for login attempts that occurred on specific dates.

```
MariaDB [organization]> SELECT *
-> FROM log_in_attempts
-> WHERE login_date = '2022-05-09' OR login_date = '2022-05-08';
```

event_id	username	login_date	login_time	country	ip_address	success
1	jrafael	2022-05-09	04:56:27	CAN	192.168.243.140	0
3	dkot	2022-05-09	06:47:41	USA	192.168.151.162	0
4	dkot	2022-05-08	02:00:39	USA	192.168.178.71	0

In the query I used it returns all login attempts that occurred between 2022-05-09 or 2022-05-08. First I started by selecting all data from the `log_in_attempts` table. After I used the WHERE clause with an OR operator to filter my results in the output to the incident occurred. The first and second conditions filter for logins on those exact dates.

Retrieve login attempts outside of Mexico

We also found that there were attempts of logins outside of Mexico. We will also have to investigate that.

The following screenshot explains how I created an SQL query to filter for logins that originated outside of Mexico.

```
MariaDB [organization]> SELECT *
-> FROM log_in_attempts
-> WHERE NOT country LIKE 'MEX%';
```

event_id	username	login_date	login_time	country	ip_address	success
1	jrafael	2022-05-09	04:56:27	CAN	192.168.243.140	0
2	apatel	2022-05-10	20:27:27	CAN	192.168.205.12	0
3	dkot	2022-05-09	06:47:41	USA	192.168.151.162	0

In the query I used it returns all login attempts outside of Mexico. First I started by using `SELECT` and `FROM` to select all data from `log_in_attempts`. Then I used the `WHERE` clause with `NOT` and `LIKE` operators, `NOT` filters for countries that are not Mexico and `LIKE` with `'MEX%'` to not search for `MEX` or `MEXICO`. For the `(%)` represents any number of unspecified characters when used with `LIKE`.

Retrieve employees in Marketing

My team wants to update the computers for certain employees in the marketing department. To accomplish this I have to sort out and get information on which machines to update.

The following code in the screenshot will demonstrate how i used SQL query to filter for employee's machines from employees in the marketing department.

```
MariaDB [organization]> SELECT *  
  -> FROM employees  
  -> WHERE department = 'Marketing' AND office LIKE 'East%';
```

employee_id	device_id	username	department	office
1000	a320b137c219	elarson	Marketing	East-170
1052	a192b174c940	jdarosa	Marketing	East-195
1075	x573y883z772	fbautist	Marketing	East-267

In my query I returned all employees in the marketing department that are in the East building. I started by using `SELECT` and `FROM` to select data from the employees table. Then, I used a `WHERE` clause with `AND` to filter for employees that work in the marketing department and in the east building. I used `LIKE` with `EAST%` to filter out all the offices in the east. I have 2 conditions in this code the first one is the `department = 'Marketing'` which filters for employees in the marketing department. The second condition is the `office LIKE 'East%'` which filters for employees in the east building.

Retrieve employees in Finance or Sales

I also had to update the machines in the finance and sales department. Since it's a different department I had to filter for information from these 2 departments.

The following screenshot shows how I used SQL query to filter out employee machines from Finance and Sales departments.

```

MariaDB [organization]> SELECT *
-> FROM employees
-> WHERE department = 'Finance' OR department = 'Sales';
+-----+-----+-----+-----+-----+
| employee_id | device_id | username | department | office |
+-----+-----+-----+-----+-----+
|          1003 | d394e816f943 | sgilmore | Finance | South-153 |
|          1007 | h174i497j413 | wjaffrey | Finance | North-406 |
|          1008 | i858j583k571 | abernard | Finance | South-170 |

```

In this query I needed to return all employees in the Finance and Sales department. I selected all data from the employees table by using SELECT and FROM. I then used a WHERE clause with OR operator to filter for employees in the Finance and sales department. The reason I use OR operator instead of AND operator is because I want to filter for employees in either department. My first condition is department = 'Finance' to filter for employees in the finance department. My second condition is department = 'Sales', which filters for employees from the sales department.

Retrieve all employees not in IT

The team needed to make one more security update on employees who are not in Information Technology.

To make this update I had to create an SQL query to filter for employee machines from employees not in the information technology department. That's what the following screenshot demonstrates.

```

MariaDB [organization]> SELECT *
-> FROM employees
-> WHERE NOT department = 'Information Technology';
+-----+-----+-----+-----+-----+
| employee_id | device_id | username | department | office |
+-----+-----+-----+-----+-----+
|          1000 | a320b137c219 | elarson | Marketing | East-170 |
|          1001 | b239c825d303 | bmoreno | Marketing | Central-276 |
|          1002 | c116d593e558 | tshah | Human Resources | North-434 |

```

In this query I used SELECT and FROM to return all employees not in Information Technology. I started by selecting all data from the employees table. Then, I used a WHERE clause with NOT to filter for employees not in this department.

Summary

In this project, I utilized the relational database tables `log_in_attempts` and `employees` to extract and analyze targeted information related to authentication activity and employee workstation usage. To achieve this, I implemented a series of filtering techniques designed to refine query results and isolate relevant data.

Specifically, I applied logical operators such as `AND`, `OR`, and `NOT` to construct precise conditional statements. These operators enabled me to combine multiple criteria, account for alternative scenarios, and exclude nonessential records. Furthermore, I employed the `LIKE` operator in conjunction with the `% wildcard` to perform pattern matching within text fields. This approach facilitated the identification of login attempts and machine identifiers that adhered to particular naming conventions or partially defined strings, rather than relying solely on exact matches.

By strategically combining logical operators with pattern-based filtering, I was able to execute queries that delivered highly specific insights. This method ensured that the extracted dataset was both accurate and contextually relevant, supporting a deeper understanding of login behaviors and system usage across the organization.